

1. Соединение таблиц

INNER JOIN (Внутреннее соединение). Выводит пересечение таблиц. Коммутативно.

```
SELECT orders.id, customers.name
FROM orders
INNER JOIN customers ON orders.customer_id = customers.id;
```

LEFT OUTER JOIN (Левое внешнее соединение). Левая таблица + правая/null.

```
SELECT customers.name, orders.order_date
FROM customers
LEFT JOIN orders ON customers.id = orders.customer_id;
```

RIGHT OUTER JOIN (Правое внешнее соединение). Правая таблица + левая/null. (Оптимизатор преобразует в left join).

```
SELECT employees.name, departments.dept_name
FROM employees
RIGHT JOIN departments ON employees.dept_id = departments.id;
```

FULL OUTER JOIN (Полное внешнее соединение). INNER JOIN + LEFT JOIN + RIGHT JOIN. Коммутативно.

```
SELECT course_a.student_name, course_b.student_name
FROM course_a
FULL JOIN course_b ON course_a.student_id = course_b.student_id;
```

CROSS JOIN (Перекрестное соединение). На каждую строку А приходятся все строки В (декартово произведение). Коммутативно. Не нужно использовать ON.

```
SELECT products.name, colors.color_name
FROM products
CROSS JOIN colors;
-- Если 10 товаров и 5 цветов, получится список из 50 сочетаний
```

NATURAL JOIN (Натуральное соединение). Работает как модификатор для предыдущих (кроме cross join) – соединяет таблицы по столбцам с одинаковыми именами. Коммутативно при inner/full.

```
-- Опасно и непредсказуемо
SELECT * FROM products NATURAL LEFT JOIN categories;
```

SELF JOIN (Самоприсоединение). Способ применения вышеперечисленных операторов через алиасы. Коммутативно при inner.

```
SELECT
    e.name AS employee_name,
    m.name AS manager_name
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

Здесь мы используем `LEFT JOIN`, чтобы не потерять самого главного босса, у которого `manager_id` равен `NULL`.

SEMI-JOIN (Полусоединение). При наличии совпадения выведет только его, не ища дальше.

```
SELECT * FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.id
);
```

ANTI-JOIN (Антисоединение). Выводит только те строки, для которых нет совпадений. Реализуется через `NOT EXISTS`, `NOT IN`, `LEFT JOIN... WHERE B.id IS NULL`.

```
SELECT * FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.id
);
```

LATERAL JOIN (Латеральное соединение). Работает как цикл. Передает левую часть в правую. Обычно используется для подзапроса/функции.

```
SELECT authors.name, top_books.title
FROM authors
CROSS JOIN LATERAL (
    SELECT title
    FROM books
    WHERE books.author_id = authors.id -- Ссылка на внешнюю таблицу!
    ORDER BY rating DESC
    LIMIT 2
) AS top_books;
```

2. Подзапросы

Подзапросы (или вложенные запросы) — это запросы типа `SELECT`, которые находятся внутри другого SQL-запроса. В PostgreSQL они позволяют строить динамические фильтры и вычислять данные «на лету» прямо во время выполнения основного кода.

По типу возвращаемых данных

- **Скалярные подзапросы:** Возвращают ровно **одно значение** (одна строка, один столбец). Их можно использовать там же, где обычную константу или переменную (например, в блоке `SELECT` или в сравнениях).
 - *Пример:* `SELECT name, (SELECT MAX(price) FROM products) FROM categories;`
- **Многострочные подзапросы:** Возвращают один столбец, но **много строк**. Используются с операторами `IN`, `ANY`, `ALL`.
- **Табличные подзапросы:** Возвращают целую таблицу. Обычно стоят в блоке `FROM` и требуют наличия алиаса (псевдонима).

2. По связи с основным запросом

Это важнейшее деление, которое напрямую влияет на производительность:

Независимые (Простые) подзапросы

Выполняются один раз перед запуском основного запроса. Результат подставляется в основной запрос как готовое значение.

Пример: Найти товары дороже среднего.

```
SELECT * FROM products WHERE price > (SELECT AVG(price) FROM products);
```

Коррелированные (Связанные) подзапросы

Внутренний запрос ссылается на колонку из внешнего запроса. Из-за этого базе данных приходится (теоретически) выполнять подзапрос **для каждой строки** внешнего запроса.

Пример: Найти товары, цена которых выше средней в их категории.

```
SELECT p1.name FROM products p1 WHERE p1.price > (SELECT AVG(p2.price)
FROM products p2 WHERE p2.category_id = p1.category_id);
```

Коррелированные запросы можно заменять `LATERAL JOIN`.

3. Представления.

Представления (Views) в PostgreSQL — это, по сути, «сохраненные запросы». Это виртуальные таблицы, которые не хранят данные сами по себе (в обычном случае), а просто показывают результат выполнения заложенного в них SQL-кода каждый раз, когда вы к ним обращаетесь.

1. Обычные представления (Regular Views)

Это самый простой тип. Когда вы делаете `SELECT * FROM my_view`, база данных «под капотом» просто подставляет код этого представления в ваш запрос.

- **Динамичность:** Данные всегда актуальны. Если данные в исходных таблицах изменились, представление покажет новые значения мгновенно.
- **Экономия места:** Они не занимают дискового пространства (кроме записи в системном каталоге).
- **Синтаксис:**

SQL



```
CREATE VIEW active_users AS
SELECT id, name, email
FROM users
WHERE status = 'active';
```

В обычные представления можно делать инсерты.

Сложные представления отличаются от обычных наличием сложных операторов.

Материализованные представления сохраняют на диск результат своего запроса. Для обновления результата нужно использовать REFRESH + имя представления.

Временные представления существуют только во время текущей сессии, видит только создатель.

Чтобы сделать инсерт в сложное представление, нужно воспользоваться либо триггерами, либо правилами.

4. Последовательности

В PostgreSQL **последовательности (Sequences)** — это специальные объекты, предназначенные для генерации уникальных числовых значений. Чаще всего они используются для создания первичных ключей (**ID**), чтобы каждая новая строка получала уникальный номер автоматически.

Самый простой способ создать последовательность вручную:

SQL



```
CREATE SEQUENCE user_id_seq
  START WITH 1
  INCREMENT BY 1;
```

Основные параметры:

- **START WITH:** С какого числа начинать.
- **INCREMENT BY:** Шаг (может быть отрицательным, если идем вниз).
- **MINVALUE / MAXVALUE:** Границы счетчика.
- **CYCLE:** Если счетчик дойдет до предела, он начнет заново с минимального значения.
- **CACHE:** Сколько чисел база заранее «забронирует» в памяти (ускоряет работу, но при сбое БД зарезервированные числа пропадут, и возникнет «дырка» в нумерации).

SERIAL (Старый стиль)

Когда ты пишешь `id SERIAL`, Postgres за кулисами делает три вещи:

1. Создает последовательность.
2. Ставит значение по умолчанию `nextval('название_последовательности')`.
3. Привязывает владение последовательностью к этой колонке.

3. Функции OWNED BY

Когда ты используешь `SERIAL`, последовательность «принадлежит» колонке. Если ты удалишь таблицу или колонку, последовательность удалится автоматически.

Если ты создал последовательность вручную через `CREATE SEQUENCE`, она «ничья».

Чтобы она удалялась вместе с таблицей, её нужно привязать:

SQL



```
ALTER SEQUENCE my_seq OWNED BY users.id;
```

3. Главные функции для работы

Чтобы управлять последовательностью вручную, используются специальные функции:

- `nextval('seq_name')` : Продвигает счетчик вперед и возвращает новое значение.
Внимание: если транзакция откатится (ROLLBACK), число всё равно будет считаться использованным. Последовательности никогда не «откатываются» назад, чтобы не было дублей.
- `currval('seq_name')` : Возвращает значение, которое ты получил в *текущей* сессии последним вызовом `nextval` .
- `setval('seq_name', 100)` : Принудительно устанавливает счетчик на нужное число (полезно после массового импорта данных).